EXECUTIVE BRIEFING

Distributed Systems: An Executive Perspective

What a distributed system buys, what it costs, and how to tell when a single machine would have done.

Vivekanand Pujar | @VivekanandPujar

Executive briefing · Decision-makers across business, technology and finance

Twenty-five slides · Boundaries, trade-offs, pitfalls and case studies

THE ROUTE

Agenda

01 **Why, and where to draw the line**
What distribution buys, how it differs from a monolith, and where business boundaries become technical ones.

02 **How it behaves**
Communication, data, availability, scale and resilience: the properties that change once the parts are separated.

03 **How to run it**
Teams, reliability engineering, observability, infrastructure as code and release management.

04 **The traps**
Overengineering, tools in place of principles, marketing, and the discipline of reasoning about the system.

05 **Getting there**
Modernising a legacy monolith, platform engineering, hosting, and the choice between building and buying.

06 **The evidence**
Four cases that worked, four that did not, and the questions to ask at the next design review.

Vivekanand Pujar | @VivekanandPujar

THE ARGUMENT

Executive summary

> **A distributed system is a purchase.** It buys independent teams, independent scaling and survivable failure. The price is partial failure, divided data and a larger operating bill. Buy it only where the business boundaries justify the price.

01

Draw lines where the business does

Boundaries follow what changes together and who answers when it breaks, not layers of technology.

02

Design for failure

The network drops, delays and duplicates. Calls, data and releases must be built to survive that.

03

Run it as a discipline

Reliability engineering, observability, automation and a platform are part of the price, not extras.

04

Earn every step

Start with a well-structured monolith, split where pain is proven, and migrate in slices, not rewrites.

Vivekanand Pujar | @VivekanandPujar

PART 1 · WHY IT DIFFERS

In a distributed system, every simple thing becomes a negotiation

CONCERN	IN A MONOLITH	IN A DISTRIBUTED SYSTEM	THE BUSINESS CONSEQUENCE
A call	A function either returns or fails, in nanoseconds	A network call may succeed, fail or vanish, and the caller cannot tell which	Retries and duplicates need designing for
A transaction	The database undoes half-finished work automatically	No system can undo work across several; the process itself must include the undo	Compensation becomes a business rule
A failure	The program works, or it does not	Parts fail while the rest carries on, in ways nobody rehearsed	"Is it up?" no longer has one answer
A diagnosis	One log, one stack trace	One request crosses many systems; the evidence is scattered	Observability becomes a budget line
A release	One build, one deployment, one rollback	Many parts on different schedules that must stay compatible	Release management becomes a function
Understanding	One team can read all the code	Nobody can hold all of it in their head	Ownership and contracts replace goodwill

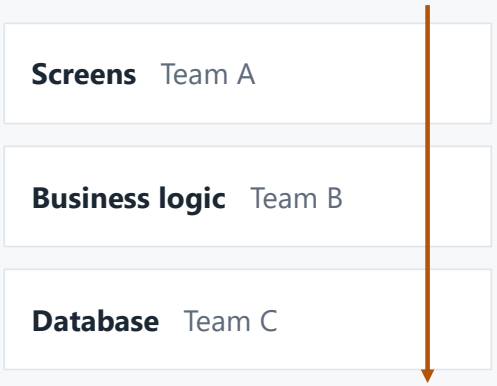
A monolith fails loudly and completely. A distributed system fails quietly and in part, which is harder to notice and to explain.

Vivekanand Pujar | @VivekanandPujar

PART 1 · THE BOUNDARY

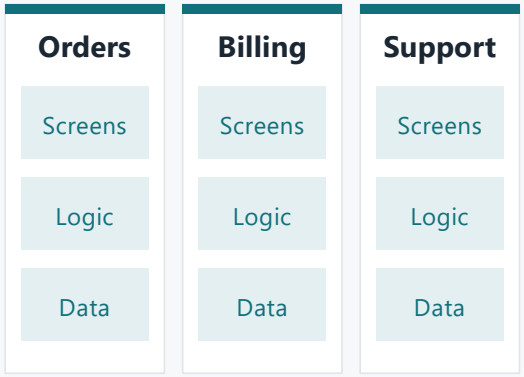
The business draws the boundaries; the technology only has to respect them

CUT BY TECHNOLOGY LAYER



One new feature crosses three teams and three queues.

CUT BY BUSINESS CAPABILITY



The same feature stays inside one team.

THREE TESTS FOR A BOUNDARY

1. **Does it change together?**
   Things that change for the same business reason belong in one place.
2. **Who answers when it breaks?**
   One team, carrying the pager, owns it from screen to database.
3. **Can it fail alone?**
   When it is down, the business should degrade, not stop.

Conway's law, 1968: organisations build systems that mirror their own communication lines. Draw the organisation first, and the architecture follows.

Vivekanand Pujar | @VivekanandPujar

PART 2 · COMMUNICATION

Every call is a promise about time: choose between asking and telling

MODALITY	EVERYDAY ANALOGY	SUITS	WHAT IT COSTS
Request and response	A telephone call	Answers needed now: a price, a card authorisation	Both parties must be up at the same moment; delays chain
Message queue	A letter to a named recipient	Work that can wait: send the invoice, resize the photo	The reply comes later; duplicates and ordering need handling
Event stream	A newspaper: publish once, anyone may subscribe	Facts that several parties care about: "order placed"	The publisher cannot know who reacts; contracts need versioning
Batch transfer	An overnight courier	Bulk work that tolerates lateness: reporting, reconciliation	Data is only as fresh as the last run

RULE OF THUMB

Ask when you cannot proceed without the answer. Tell when others merely need to know.

THE TRAP

A chain of synchronous calls is only as available as all of its links multiplied together.

Vivekanand Pujar | @VivekanandPujar

PART 2 · DATA

Exactness is the expensive part of distributed data, so buy it fact by fact

Must be exact

Strong consistency

TYPICAL FACTS

Account balances, stock at the till, a seat on a flight.

WHAT IT COSTS

Every write waits for copies to agree. Slower across regions.

Soon enough

Eventual consistency

TYPICAL FACTS

The catalogue, an order-status page, the search index.

WHAT IT COSTS

Copies update in the background. Two users may briefly see different answers.

Roughly right

Approximate, cached

TYPICAL FACTS

View counts, recommendations, a management dashboard.

WHAT IT COSTS

Cheapest and fastest. A number a minute stale harms nobody.

WHEN THE NETWORK SPLITS

A system must either refuse requests or answer and risk disagreement. Decide which, fact by fact, before it happens.

WHERE THERE IS NO UNDO

Airlines overbook and then pay vouchers. A distributed system asks everyone else to write the apology into the process.

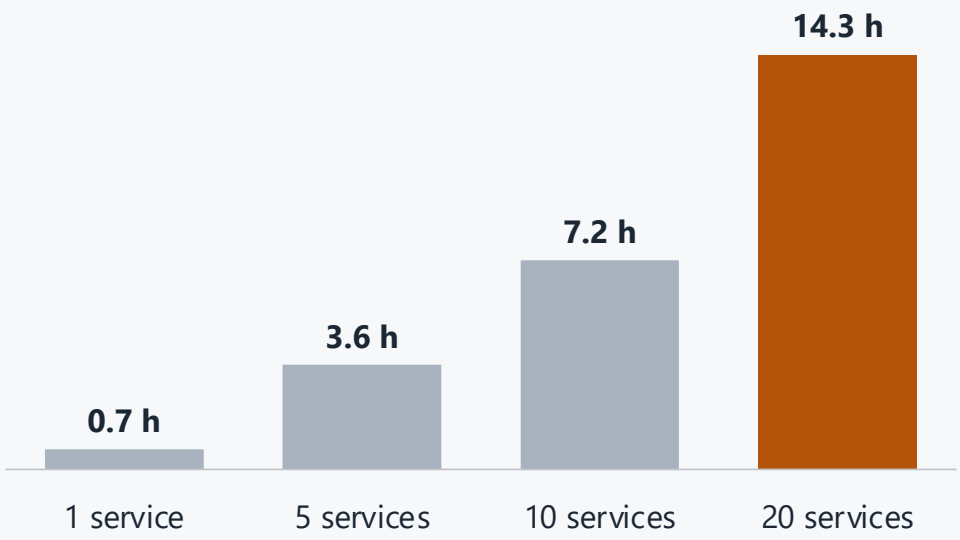
Vivekanand Pujar | @VivekanandPujar

PART 2 · AVAILABILITY AND SCALE

Availability multiplies down the chain, and scale is limited by the narrowest point

Expected downtime per month, hours

Chain of services, each 99.9% available



THE INTRICACIES OF AVAILABILITY AND SCALE

Redundancy is not availability

A second copy helps only if failover has been rehearsed. An untested failover is a hypothesis.

Servers scale; state does not

Adding machines is easy. Dividing the data is hard, and one hot customer can cap the whole system.

Capacity is set by the narrowest point

One database, one queue or one partner interface decides the ceiling for everything upstream.

Autoscaling takes minutes

Spikes arrive in seconds. Headroom, queues and load shedding cover the gap.

Assumes independent failures and that every call in the chain is required; 30-day month. Availability of the chain is the product of its parts.

Vivekanand Pujar | @VivekanandPujar

PART 2 · RESILIENCE

Resilience is designed behaviour below full health, not a hope that nothing breaks

Timeouts

Do not wait forever

A slow answer is treated as no answer once the time budget is spent.

Limited retries

Redial, but sparingly

Three tiers retrying three times each send 27 requests to the one already struggling.

Bulkheads

Watertight compartments

One customer's surge cannot starve the rest. The Titanic's did not reach the top deck: check yours.

Circuit breakers

A fuse in the wall

Stop calling a failing partner, test it now and then, and resume when it recovers.

Graceful degradation

Open with half the lights on

Show cached prices, hide recommendations, queue the order. Decide in advance what goes first.

Rehearsal

Fire drills for software

Game days and fault injection, as in Netflix's Chaos Monkey, find the weak joint before customers do.

PART 3 · TEAMS

The architecture will copy the organisation, so organise first

STREAM-ALIGNED TEAMS: ONE BUSINESS OUTCOME EACH

Orders	Billing	Support
builds it, runs it	builds it, runs it	builds it, runs it

PLATFORM TEAM: THE PAVED ROAD

Pipelines, runtime, observability, security
Run as a product whose customers are the other teams

ENABLING TEAMS: COACHING, NOT CONTROLLING

Reliability engineers	Architecture and security

WHAT CHANGES IN THE ORGANISATION

- **You build it, you run it.** The team that ships carries the pager (Werner Vogels, 2006).
- **Cognitive load is the limit.** A team should own no more than it can hold in its head.
- **Interfaces are contracts.** Versioned and tested like products, because other teams depend on them.
- **Fund outcomes, not projects.** A service outlives the project that built it.

Rule of thumb: a team small enough to share two pizzas is small enough to understand its own service.

Vivekanand Pujar | @VivekanandPujar

PART 3 · RELIABILITY ENGINEERING

Reliability is a feature with a budget, and someone must own it

TARGET	DOWNTIME A MONTH	WHAT IT TAKES
99%	7 hours 12 minutes	Working-hours cover and manual recovery
99.9%	**43 minutes**	Automated releases, alerting and an on-call rota
99.99%	4 minutes 19 seconds	Several zones, automated failover, rehearsed
99.999%	26 seconds	Several regions and no human in the loop

Each extra nine costs more than the last and is worth less to the customer. Choose the one the customer will notice.

HOW RELIABILITY ENGINEERING WORKS

1 **Agree an objective**

"99.9% of checkouts succeed within two seconds." Business and engineers sign the same number.

2 **Spend the error budget**

Inside the budget, ship fast. Once it is spent, features wait while reliability is repaired.

3 **Learn without blame**

Post-incident reviews change the system, not the rota.

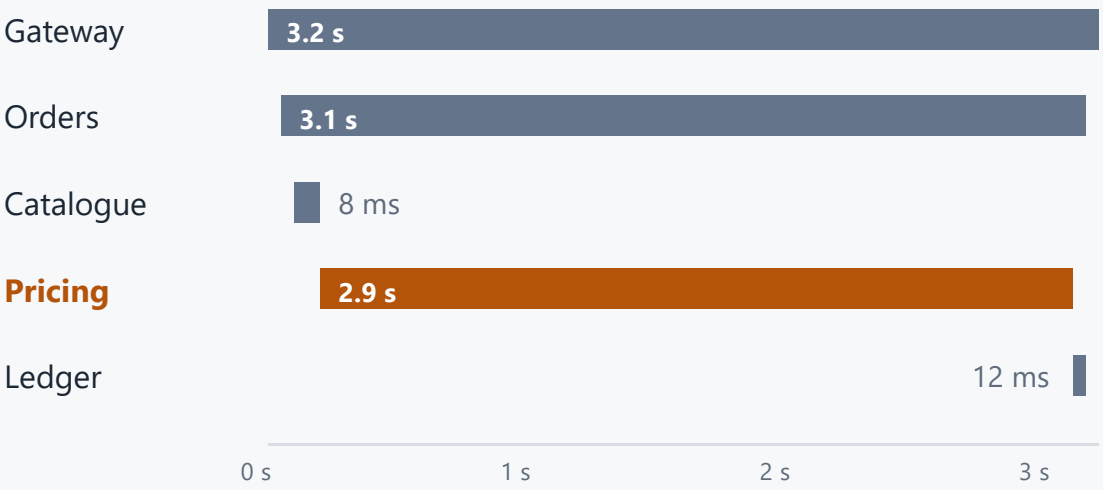
Vivekanand Pujar | @VivekanandPujar

PART 3 · OBSERVABILITY

Monitoring says that something is broken; observability says why

One checkout request, followed through five systems

A trace: each bar is one call, drawn against time



THREE SIGNALS, THREE QUESTIONS

Metrics *Is something wrong?*

Counts and rates, like the warning light on a dashboard.

Logs *What did it say?*

Records of events, like the black-box recorder.

Traces *Where did the time go?*

One request across all systems, like a parcel's tracking number.

The test: can the engineer on call explain a failure nobody predicted, without shipping new code? Telemetry is also a budget line, so sample on purpose.

Vivekanand Pujar | @VivekanandPujar

PART 3 · INFRASTRUCTURE AND RELEASES

A system of many parts must be built, provisioned and released by machine

Infrastructure as code

Environments written down, not remembered

- Every environment is built from reviewed files, so test and production match.
- Drift raises an alert instead of surfacing in an audit.
- The change history is the audit trail regulators ask for.
- Rebuilding the whole estate in another region is a command, not a project.

Release management

Small, frequent and reversible

- Small releases make a failure easy to find.
- Canary releases and feature flags limit how many customers see a bad change.
- Rollback is rehearsed, not improvised.
- Release and launch become separate decisions: flags let the business choose the day.

FOUR NUMBERS THAT MEASURE DELIVERY HEALTH (THE DORA METRICS)

Deployment frequency
How often a change reaches production

Lead time
From commit to running in production

Change failure rate
Share of releases that cause an incident

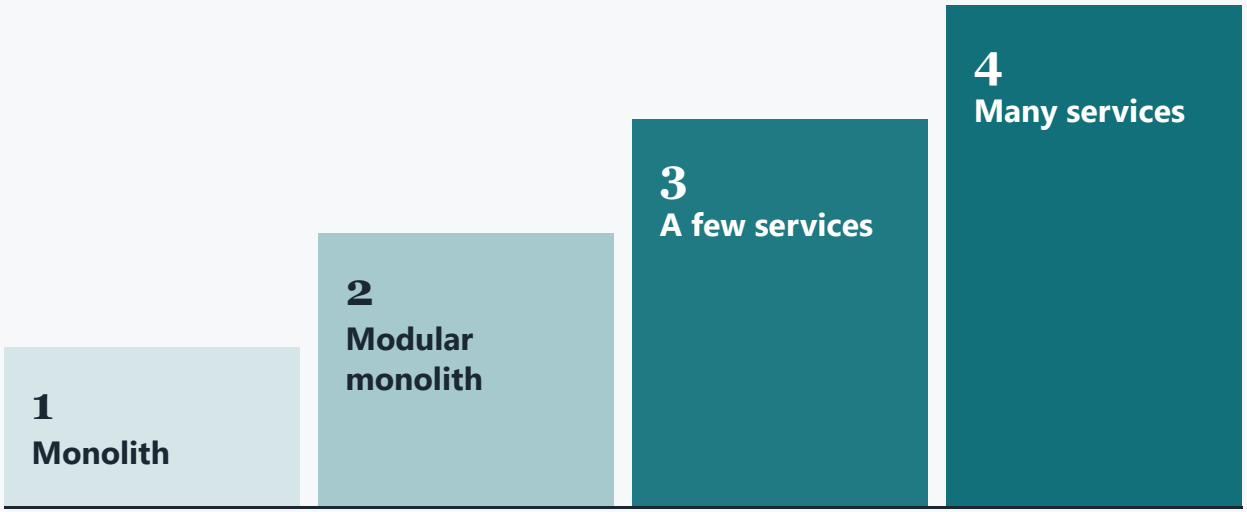
Time to restore
How fast service returns after one

Vivekanand Pujar | @VivekanandPujar

PART 4 · OVERENGINEERING

Most systems that were distributed did not need to be

EACH RUNG ADDS OPERATING COST AND ORGANISATIONAL DEMAND



REASONS THAT SURVIVE THE CFO

- Separate teams need to release on their own.
- Parts have very different load or uptime needs.
- Security or regulation demands isolation.
- One part needs different technology, such as a search engine.

REASONS THAT DO NOT

- *"It is modern."*
- *"A famous firm does it."*
- *"It will look good on the CV."*

Vivekanand Pujar | @VivekanandPujar

PART 4 · TOOLS AND PRINCIPLES

A tool answers a question; first make sure someone has asked it

THE PRINCIPLE	THE TOOL USUALLY BOUGHT	THE QUESTION THE TOOL DOES NOT ANSWER
Isolate failure	Kubernetes, a service mesh	What does the product do when this dependency is down?
Release independently	Containers, a delivery suite	Can team A ship without waiting for team B?
Understand behaviour	A monitoring suite	Can the engineer on call explain a failure nobody predicted?
Own the data	An event platform, a data lake	Who owns this fact, and who decides what it means?
Change safely	A feature-flag product	Who decides to switch it on, and who switches it off?

Buying the tool first is like buying a gym membership and calling it fitness. The tool is the receipt; the principle is the habit.

PART 4 · MARKETING

“Microservices” is a brand; distributed systems are the discipline

THE LABEL	WHAT IT USUALLY MEANS	THE QUESTION TO ASK
Microservices	A distributed system built from many small parts, with all the old problems: failure, delay, divided data	How many teams, and can each release without the others?
Serverless	Servers run by someone else and billed by use	What does it cost at our steady load, and where are its limits?
Cloud-native	Designed to expect failure, which is not the same as hosted in the cloud	What happens when one data centre disappears?
Event-driven	Asynchronous messaging, in use since before the web	Who owns the meaning of each event, and how is it versioned?
Zero downtime	A promise no real system keeps	Which number of nines, measured how, at what price?

The field is older than the label. Lamport’s 1987 joke, that a computer you never heard of can stop yours, and the 1990s fallacies of distributed computing still hold.

Vivekanand Pujar | @VivekanandPujar

PART 4 · REASONING

The scarce asset is the ability to reason about the system, not the number of tools

ONE MENTAL MODEL, WORKED THROUGH

In flight = rate × time

Little's law: the work inside a system equals the arrival rate times the time each item stays.

On a normal day

100 requests a second × 0.2 s = 20 in flight.

With one slow dependency

100 × 2 s = 200 in flight. A pool of 200 workers is full, and healthy services behind it are unreachable.

HOW TO CULTIVATE THE ABILITY

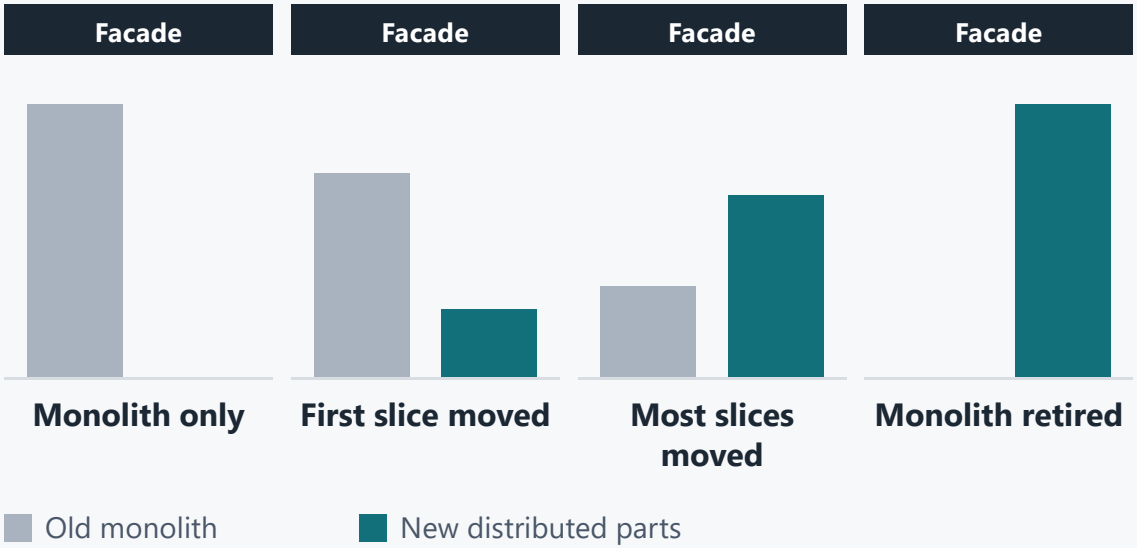
1. **Put failure in every review**
   Ask what happens when each part is slow, down or wrong.
2. **Share a few mental models**
   Queues, quorums, retries and consistency, in plain terms.
3. **Rehearse**
   Run game days before customers run them for you.
4. **Review incidents without blame**
   Publish the findings; change the system, not the rota.
5. **Put designers on call**
   Nothing improves a design like being woken by it.

Vivekanand Pujar | @VivekanandPujar

PART 5 · MODERNISING A MONOLITH

Strangle the monolith slice by slice; do not replace it in one move

ILLUSTRATIVE: WHERE THE WORK IS SERVED, STAGE BY STAGE



The pattern is named after the strangler fig, which grows around a tree until the tree is no longer needed.

1. **Find the seams**
   Organise the code by business capability, inside the monolith first.
2. **Put a facade in front**
   One routing point, so callers never notice the move.
3. **Move one slice at a time**
   Begin where change or pain is greatest, with a way back.
4. **Move the data last**
   A shared database is the usual trap; split it deliberately.
5. **Switch off the old part**
   A slice is done only when the old code is deleted.

Vivekanand Pujar | @VivekanandPujar

PART 5 · WHY IT FAILS, HOW IT SUCCEEDS

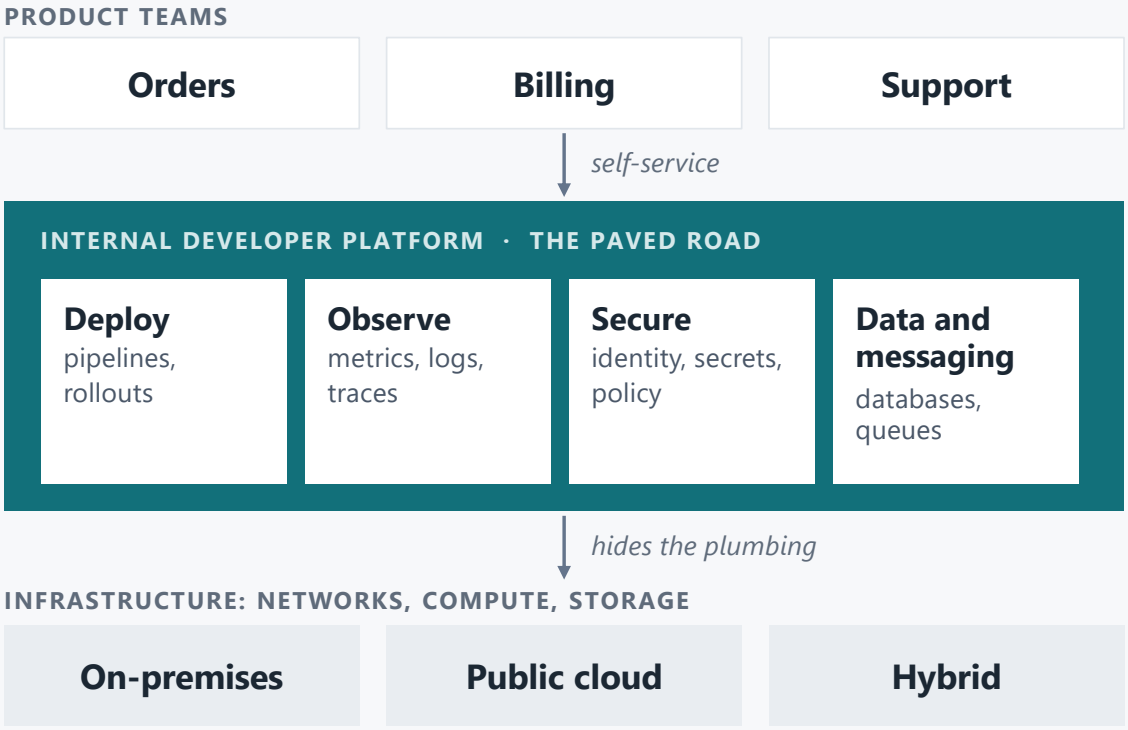
Modernisations fail for organisational reasons and succeed by being dull

WHERE IT GOES WRONG	WHAT THE SUCCESSFUL ONES DO
A from-scratch rewrite; Netscape went about three years without a release	Slices that ship every few weeks, each with a way back
The goal is “adopt microservices”	The goal is a business number: lead time, outage cost, cost of change
Parts that must release together: a distributed monolith	Each slice deploys alone, and the team proves it
The database stays shared, and nobody owns it	Each fact has one owner; the data is split on purpose
The organisation stays the same and only the boxes change	Teams are reorganised around the slices first
The old system is never switched off, so the bill doubles	Decommissioning dates are agreed at the start

Technology is rarely the cause. Scope, sequence and ownership usually are.

PART 5 · PLATFORM AND INFRASTRUCTURE

A platform team turns the hard parts into a paved road



FOUR RULES FOR A PLATFORM TEAM

Run it as a product

Developers are the customers. Measure adoption, not compliance.

Make the paved road the easy one

Teams may leave it, but then they own what they build.

Keep it thin

It hides plumbing, not decisions about the business.

Start when teams duplicate work

Several teams building the same pipeline is the signal.

Vivekanand Pujar | @VivekanandPujar

PART 5 · WHERE TO RUN IT

On-premises, cloud, hybrid or multi-cloud: decide on constraints, not fashion

OPTION	CHOOSE IT WHEN	THE HIDDEN COST
On-premises	Load is steady, data must stay in the building or country, and the hardware is paid for	Capacity is planned in advance; growth and renewal are slow and lumpy
One public cloud	Demand is spiky or uncertain, speed matters more than unit cost, and the operations team is thin	Exit fees and lock-in; the bill grows unless someone owns it
Hybrid	Regulated data stays put while new products run in the cloud, and migration will take years	Two operating models, a critical network link and twice the skills
Multi-cloud	A regulator demands an exit plan, an acquisition brought a second cloud, or one service is genuinely the best	The lowest common denominator of both, and rarely more resilient than several regions of one

Multi-cloud is a strategy for the board's comfort more often than for the system's. Decide on data, demand and skills.

Vivekanand Pujar | @VivekanandPujar

PART 5 · BUILD, ADOPT OR BUY

Buy what is common, build what differentiates, and price the people either way

OPTION	BEST FOR	SPEED	CONTROL	UPKEEP	MAIN RISK
Vendor SaaS	Common needs: payroll, email, payments, identity	**Fast**	**Low**	**Low**	The vendor holds the price and the roadmap
Open source, self-run	Mature parts where data and cost control matter: databases, queues, search	**Medium**	**High**	**High**	You carry the pager and the patches; free is not free
Managed open source	Open standards without the operating load	**Fast**	**Medium**	**Medium**	Proprietary extras creep in and portability fades
Custom build	What makes the firm different: pricing, matching, underwriting rules	**Slow**	**Full**	**High**	Maintenance never ends, and the team becomes the dependency
Customised package	Rarely: when regulation or an existing estate forces it	**Slow**	**Medium**	**High**	Every customisation taxes every upgrade

Buy the commodity, build the differentiator, and cost the people either way.

Vivekanand Pujar | @VivekanandPujar

PART 6 · CASES THAT WORKED

Success came from a clear reason and a patient method, not from the label

Netflix 2008 to 2016

A database corruption halted DVD shipments for three days. Netflix moved to distributed services on AWS, finishing in 2016.

WHY IT WORKED A clear trigger, seven patient years, and resilience practice built alongside.

Amazon Early 2000s

Teams were told to expose their work as service interfaces and to run what they built.

WHY IT WORKED The organisation changed first; the architecture followed it.

Shopify Modular monolith

Millions of lines in one codebase, with component boundaries enforced by tooling rather than by a network.

WHY IT WORKED Most of the benefit of boundaries, none of the network tax.

Prime Video 2023, monitoring tool

A video-quality monitor built from serverless parts was folded into one process. Infrastructure cost fell by over 90%.

WHY IT WORKED The team measured, saw that distribution added cost, and reversed it.

Sources: company engineering blogs and press reports. Amazon's service mandate is widely reported rather than officially published.

Vivekanand Pujar | @VivekanandPujar

PART 6 · CASES THAT FAILED

Failure came from untested change, unowned wholes and boundaries drawn in the wrong place

Knight Capital 2012

New trading code reached seven of eight servers; the eighth still ran old code. The firm lost over $400 million in about 45 minutes.

THE LESSON Release management is a risk control: manual deployment, dead code and no kill switch.

TSB 2018

5.2 million customers were cut over in one weekend. Up to 1.9 million lost online access; the cost passed £330m, plus a £48.65m fine.

THE LESSON A big-bang cutover has no way back. Test at full load and migrate in stages.

Healthcare.gov 2013

The US health exchange launched from many contractors' parts that had not been tested together, and failed under load.

THE LESSON Nobody owned the whole. End-to-end testing and one accountable integrator were missing.

Segment Destinations pipeline

One service per destination grew past 140; three engineers mostly kept them alive. The team merged them back into one.

THE LESSON The split multiplied the work faster than it divided it. The boundary was wrong.

Sources: SEC order (2013); FCA and PRA final notice (2022); US GAO (2014); Segment engineering blog (2018).

Vivekanand Pujar | @VivekanandPujar

CLOSING

Questions for the next design review

BOUNDARIES

01 Which business capability does each part serve, and who answers when it breaks?

02 Does any planned feature need two teams to release together?

03 What evidence says a split is needed, rather than a well-organised monolith?

DATA AND FAILURE

04 Which facts must be exact, and which may be minutes old?

05 What does the product do when each dependency is slow or down?

06 Which two systems does one operation write to, and what makes that safe?

RUNNING IT

07 Can the engineer on call explain a failure nobody predicted?

08 How is the environment built, and how do we go back from a bad release?

GETTING THERE

09 Which business number will this change move, and by when?

10 Which old part will be switched off, and on what date?

> **A distributed system is a purchase. Buy it for a reason, and know what it costs.**

Vivekanand Pujar | @VivekanandPujar